What the frozen forests split on

A reading of the two random forests frozen on 12 October 2026. The trees were opened and counted. They were not retrained.

Stage 2 asks whether memory utilization or power is moving. If it is, the window is called training. Fallback asks whether memory jumped in the first 30 samples of the run. If it did, the window is called training. Two of fallback's later questions point the other way: a GPU that is nearly full, and a memory bus that is busy while the processors are not. The tables list each question, where it sits in the tree, and which side is called training. The last section turns those cuts into adversarial workloads: each job is built to sit on the side that hurts the detector.

What we did

The detector has a network rule and two forests on the same 174 GPU features. The rule fires when a node's mean send+receive rate is at least 1 MiB/s and the two directions are balanced (the smaller is at least 0.8 of the larger). Stage 2 was fit only on windows where that rule fires: 3,019 training windows and 939 others, out of 3,958. Fallback was fit on every honest window: 10,059 windows, 244 jobs, 42 families. A window is called training when the forest that is allowed to speak has probability at least 0.5. Adversarial jobs were not in the fit.

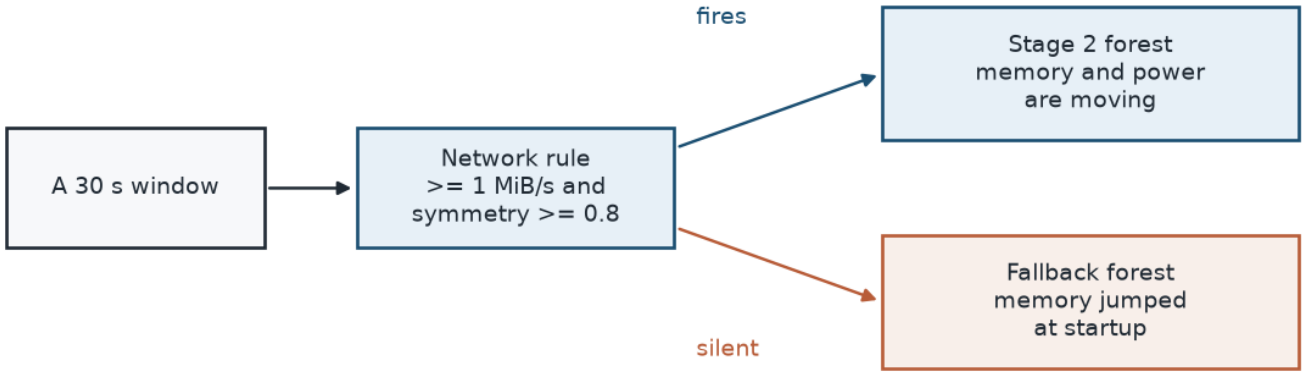


Figure 1. Which forest is asked. Once the rule fires, fallback is not consulted.

Each forest is 200 trees, minimum 2 samples in a leaf, 13 features drawn at each split, class weights balanced, seed 0. The stage 2 and fallback sections list the questions those trees ask. A cut quoted there is the typical threshold, the median of many split points weighted by how much mix each split removed. "Where the impurity goes" says how the bar chart counts those questions.

The two forests are not the same size

Stage 2	Fallback
Mean depth 4.8, deepest 9. About 10 leaves per tree. 162 of 174 features are split on at least once.	Mean depth 13.1, deepest 18. About 78 leaves per tree. 164 of 174 features are split on at least once.

Stage 2 is a short tree. Fallback is a real one. They also cut on different facts, which is the rest of this note.

Where the impurity goes

Skip ahead to Stage 2 if you want the questions first. This section is only how the bar lengths were counted.

Each tree is a stack of yes/no questions. A question looks at one column ("is the spread of memory utilization above 1.2?") and sends the window left or right.

Before the question, training and not-training are mixed together. Gini measures that mix. With two classes it is 2p(1-p), where p is the share of the weight that is training. It equals 0.5 when the two classes are even, and 0 when every window on that side is the same class. A useful question is one that makes the mix drop. The drop at a node is

```
N * Gini - N_left * Gini_left - N_right * Gini_right
```

N is not a count of windows. The forest was fit with balanced class weights, so the scarcer class pulls harder than its share of the rows, and N is that weight, already stored on the node. We add the drop over every question in all 200 trees and give the sum to the column the question used. Divide by the total drop. That percentage is the bar. It is the same number sklearn calls feature importance. Recomputing it from the nodes matches sklearn to 0.0002.

The bars are not 174 separate columns. The spread, the range, and the interquartile range of memory utilization are one measurement saved three times. Drawn separately, one fact would look like three. Columns that come from the same place are added:

Bar	Which columns go into it
Memory	The name contains "mem": utilization, megabytes in use, and the jump in the first 30 samples.
Power and temperature	The name contains "power" or "temperature".
PCIe	The name starts with "pcie".
Compute	Whatever is left: GPU utilization, SM clock, idle fraction, tensor activity.
Video	Encoder and decoder. Those columns do not vary here, so the share is zero.

A column scores only at questions where the tree actually picked it. Each question is shown 13 of the 174 columns, so a twin that was not shown gets nothing for that question. And stage 2's memory bar of 61% does not mean memory caused the call. The startup jump and the utilization wobble are both inside that bar. Figure 2 is the total. The two tables below split that bar into the actual questions.

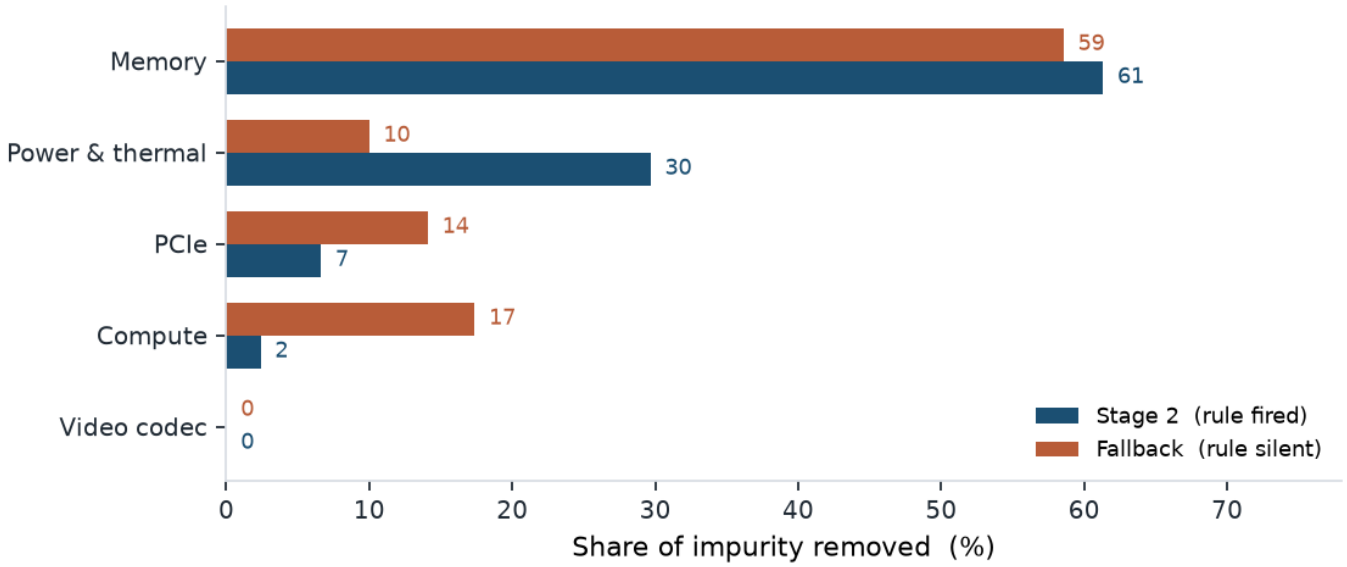


Figure 2. Of all the mix a forest removed, the share removed by questions about each source. Stage 2's memory bar is 61%: that share of the mix it removed came from a split on a column whose name contains "mem".

Stage 2 puts 61% on memory and 30% on power and temperature. PCIe is 7%. Compute, which is where GPU utilization and SM clock live, is 2%, and none of that is at the root. GPU-utilization mean is rank 105 and removes essentially no impurity.

Fallback is still 59% memory, but the memory it uses is the allocation at the start of the run. Compute rises to 17%, PCIe to 14%, and power falls to 10%.

Stage 2: is memory or power moving?

Stage 2 trained only on windows where the network rule had already fired: 3,019 training windows and 939 others. The forest is short, average depth 4.8, about 10 leaves per tree. The not-training windows in that fit are HPC jobs with balanced traffic and a flat GPU trace. Inference was not in the fit. A serving window whose memory utilization moves lands on the training side of every question below, which is why stage 2 never grew a leaf that recognizes inference.

The question	Where in the tree	Above the cut
Is the spread of memory utilization above 1.2 percentage points?	First question in 17 of 200 trees. 88% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.95 higher than below it, so the two sides are nearly pure.
Is the range of that same utilization above 4.2 percentage points? Same measurement, stored again.	First question in 17 of 200 trees. 92% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.95 higher than below it, so the two sides are nearly pure.
Is the interquartile range of that same utilization above 1.3 percentage points? Same measurement, a third copy.	First question in 10 of 200 trees. 94% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.90 higher than below it, so the two sides are nearly pure.
Is the spread of power above 9 W?	First question in 13 of 200 trees. 92% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.86 higher than below it, so the two sides are nearly pure.
Is GPU memory in use above 10 GB?	First question in 11 of 200 trees. 74% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.88 higher than below it, so the two sides are nearly pure.
Does memory take longer than 16 samples to reach its plateau? A slow climb is above this cut.	First question in 12 of 200 trees. 81% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.90 higher than below it, so the two sides are nearly pure.

Table 1. Stage 2's questions. "Above the cut" is the side where the measured value is higher. The training-share difference is the tree's own class mix on the high side minus the mix on the low side, weighted by how much mix the split removed. A difference of 1 would mean one side is all training and the other side is all not-training. Depth 0 is the first question in a tree.

The startup jump that dominates fallback ranks between 32 and 64 here, so stage 2 barely asks it. GPU-utilization mean ranks 105. Spread, range, and interquartile range together are the first question in 44 of the 200 trees. Each split is shown only 13 of the 174 columns, so a measurement that is the first question about as often as it is offered is one the tree actually prefers.

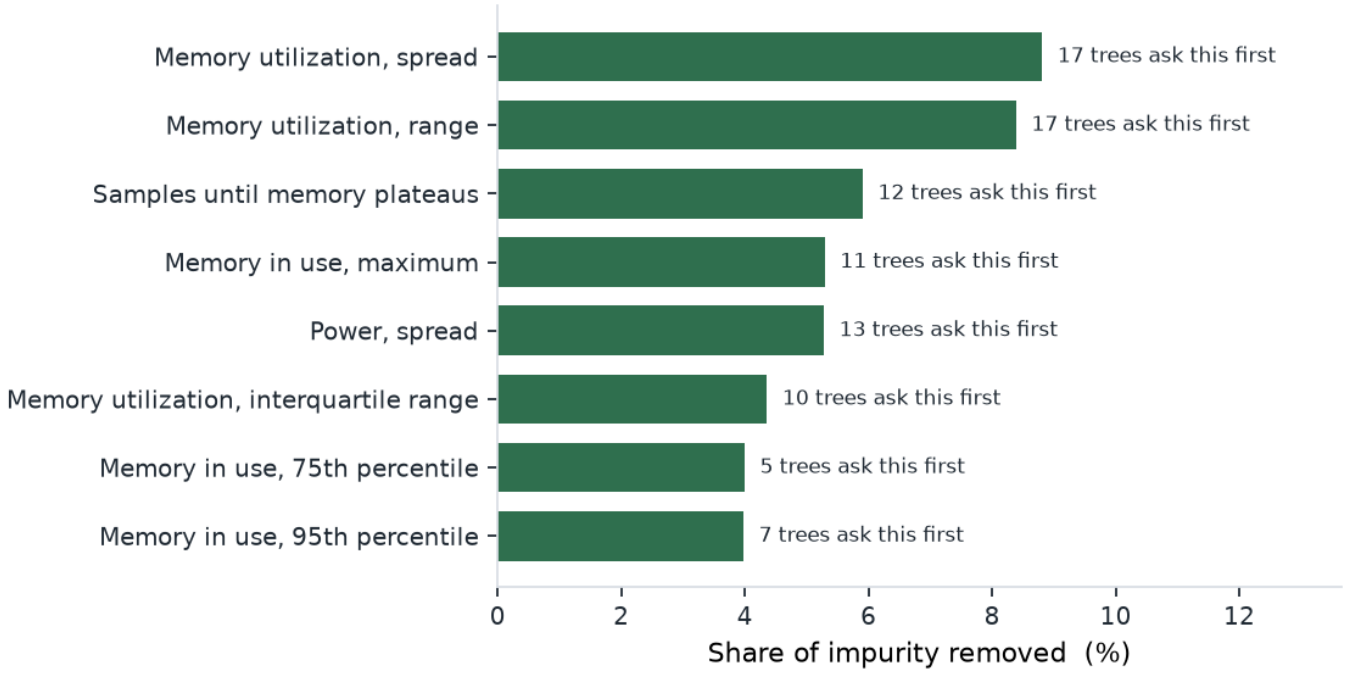


Figure 3. The same stage 2 columns, sized by how much mix they remove. Green means above the cut is training. Several bars are one measurement: spread, range, and the interquartile range of memory utilization.

Fallback: did memory jump at startup?

Fallback trained on every honest window. It is a deeper forest, average depth 13.1, about 78 leaves per tree. Its first questions read the first 30 samples of the run, and only as far as the current window is allowed to see. Once those 30 samples have passed, later windows of the same job all see the same jump. The three startup rows below are that one jump written three ways. Together they are 20% of the mix fallback removes, and 37% of the weight of first questions. The code comment says the feature was written to catch optimizer state allocated at startup, on the order of 6 GB.

The question	Where in the tree	Above the cut
Did memory rise by more than 4.9 GB in those first 30 samples?	First question in 11 of 200 trees. Most of the mix it removes comes from one or two questions later.	Training. Above the cut, the training share is 0.70 higher than below it.
Is the peak in those samples more than 11 times the starting value? Same jump, as a ratio.	First question in 15 of 200 trees. 45% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.68 higher than below it.
Is that rise more than 4% of the GPU's memory? Same jump, as a fraction of capacity.	First question in 13 of 200 trees. 36% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.63 higher than below it.
Is memory utilization minus GPU utilization above -23 points? Above this, the memory bus is busy relative to the processors.	First question in 12 of 200 trees. 36% of the mix this column removes comes from that first question.	Not training. Above the cut, the training share is 0.62 lower than below it.
Is GPU memory in use above 68 GB? Stage 2's cut on this same column is 10 GB, and there the high side is training.	First question in 8 of 200 trees. Most of the mix it removes comes from one or two questions later.	Not training. Above the cut, the training share is 0.36 lower than below it.
Is the range of PCIe transmit above 859 MB/s?	First question in 13 of 200 trees. 48% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.57 higher than below it.

Table 2. Fallback's questions, read the same way as Table 1. The fourth and fifth rows point the other way: above the cut is not training.

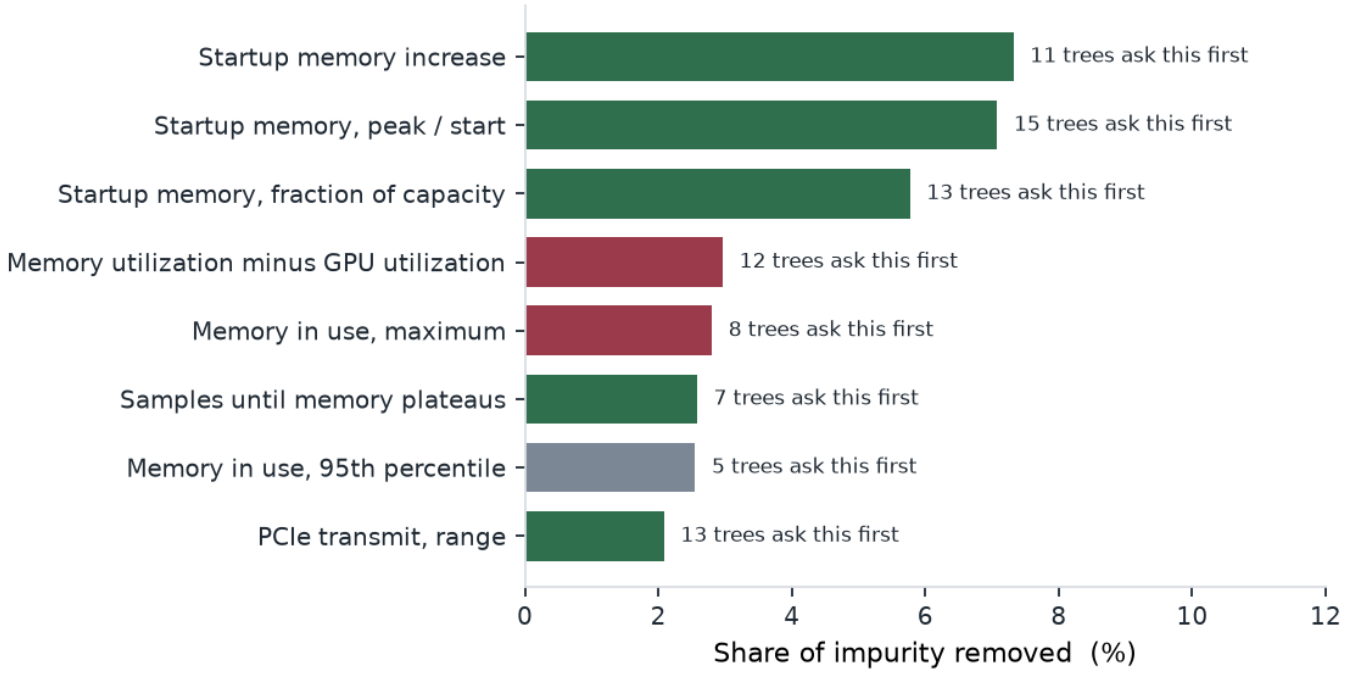


Figure 4. The same fallback columns as Table 2, sized by how much mix they remove. Green: above the cut is training. Red: above the cut is not training. Gray: the two sides barely differ. The first three bars are one jump, written three ways.

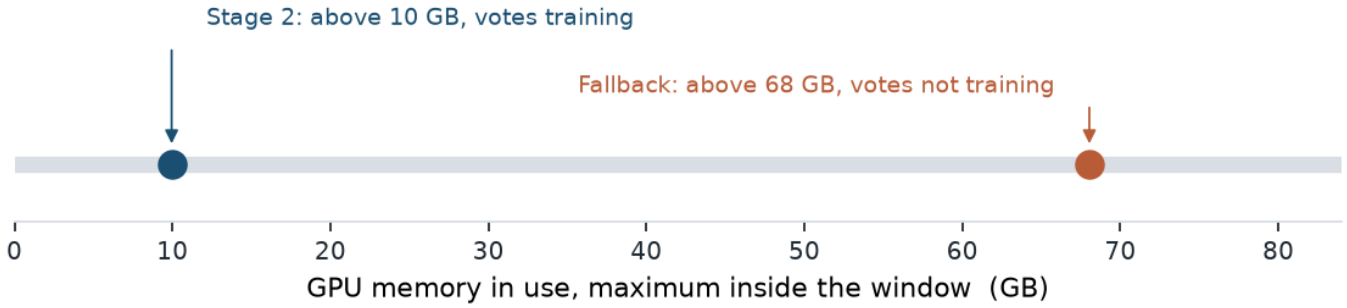


Figure 5. One column from Table 2, the two forests disagree on it. Stage 2 calls training above about 10 GB. Fallback calls not-training above about 68 GB. Both cuts are weighted medians of split points.

GPU utilization is not one of these questions. In fallback its mean ranks 19, and the typical cut is 99.7%, so only a pinned GPU registers. The training share above that cut is 0.52 higher than below it, a weak split. In stage 2 this column ranks 105.

One stage 2 tree, and one window walking through it

Stage 2 is 200 trees, not one tree. The drawing is tree 30, one of the 17 whose first question is the spread of memory utilization. It has 10 leaves. A number of windows on a leaf is how many training-fit windows reached it. The blue path is one 30-second window of Qwen serving plus a fake all-reduce. Its spread is 4.7 points, its memory kurtosis is 0, and its memory in use is 67 GB. The rule has fired on this window: about 81 MiB/s, symmetry 1.00.

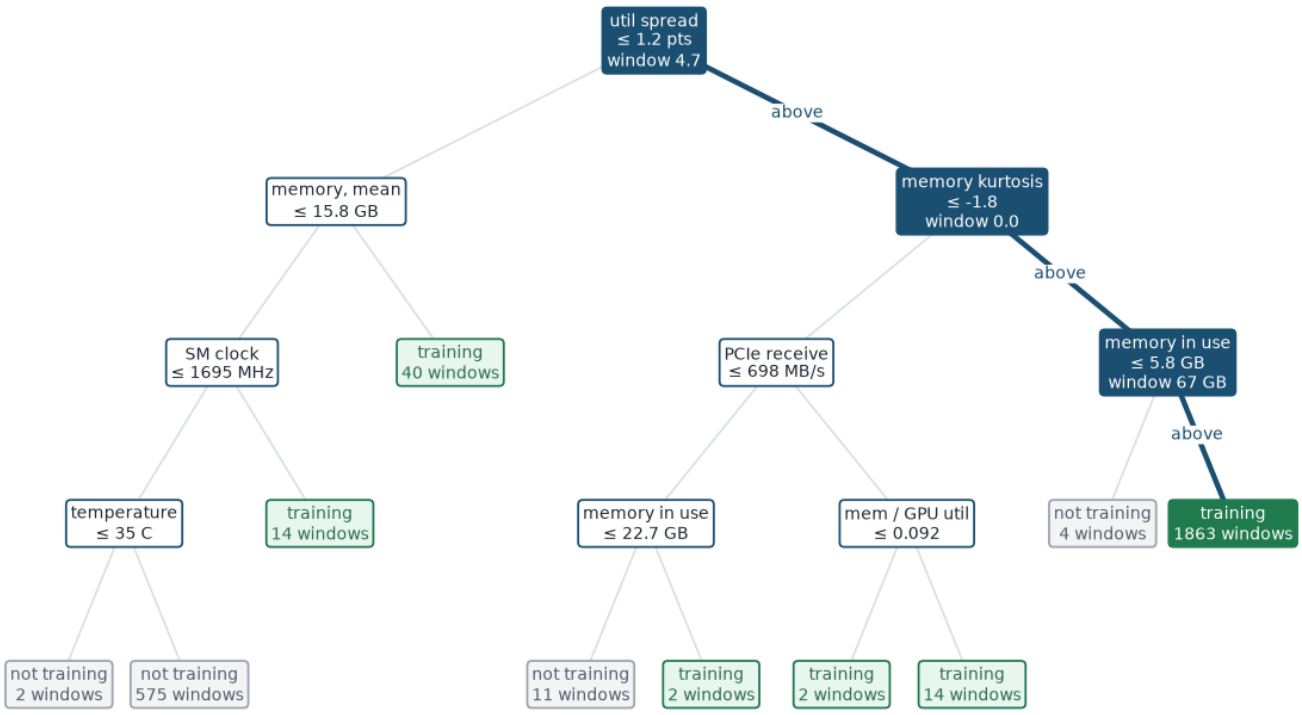


Figure 6. Every node of stage 2 tree 30. Below means the measured value is under the cut, above means it is over the cut. The window takes the blue path and ends in the training leaf that holds most of the windows this tree saw. The cuts on this one tree are not the typical cuts in Table 1. This tree asks about memory in use at 5.8 GB. The typical cut across all 200 trees is about 10 GB.

The kurtosis question peels off only a few dozen windows. Almost every window with spread above 1.2 points continues, and a memory footprint above 5.8 GB is then called training. On this window, 187 of the 200 trees vote training (probability 0.94). Fallback's probability on the same window is 0.32, and it is not asked, because the rule fired.

One fallback tree, the first three levels

A fallback tree has about 78 leaves, so the whole tree does not fit on a page. This is tree 0, one of the 15 whose first question is the startup peak divided by the start. Only the first three levels are drawn. The blue path is one 30-second window of Training with InfiniBand off. The ratio is 10508, because the feature floors the starting memory at 1 MB and this run's first samples are near empty while the peak is about 10 GB. The rule is silent: about 0 MiB/s on the wire, so this forest is the one that gets asked.

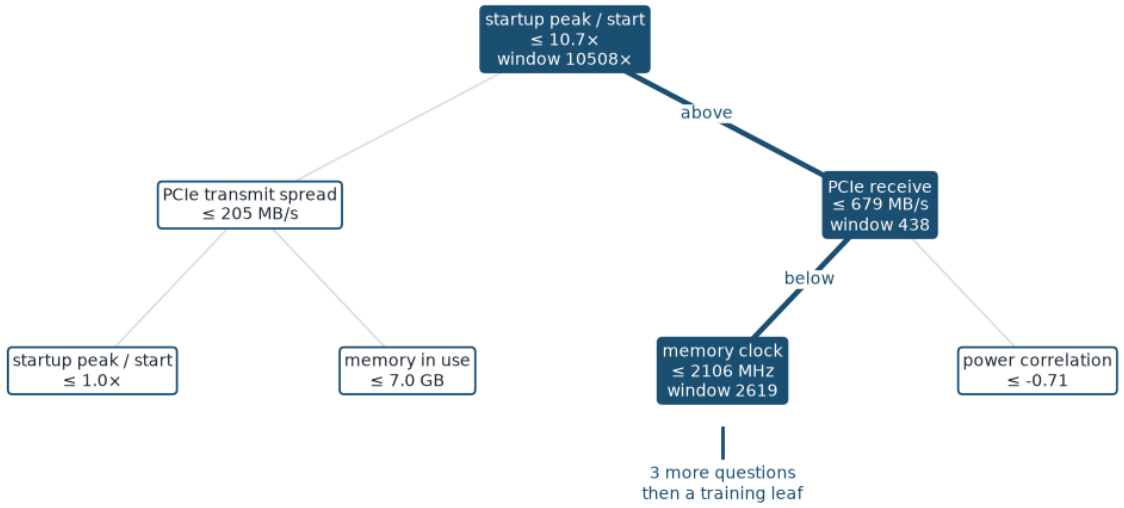


Figure 7. The top of fallback tree 0. The window goes above the startup cut, then below the PCIe cut, then through three more questions (memory clock, then temperature twice) and reaches a leaf that says training. Those three questions are the kind of fine split a deep tree grows. They are not the questions the forest leads with.

On this window, 127 of the 200 fallback trees vote training (probability 0.63). One tree reaching a training leaf is not the forest's vote. The vote is how many of the 200 agree.

What that does to the gate

On the 93 Nebius jobs already scored with this frozen detector, the pattern matches the cuts. Those jobs were not used to choose features or to move the 0.5 cutoff. Probabilities below are the mean over the job's windows.

Workload	Rule	Stage 2	Fallback	Call
Training, IB off (iboff_flat)	silent	not asked	0.65	training
Training, no gradient collective (nosync_flat)	silent	not asked	0.62 to 0.66	training
LoRA, one node, IB off (lora_1n8_iboff)	silent	not asked	0.53 to 0.54	training on 85% of windows
Qwen serving plus a fake all-reduce (qwen_fake_ar)	fires	0.90 to 0.93	0.30 to 0.37	training
OLMoE across nodes (olmoe_ep_mixed)	fires	0.89 to 0.90	0.11 to 0.13	training
Split prefill/decode plus a return message (disagg_echo)	fires on most windows	0.91 to 0.95	0.35 to 0.47	training

Table 3. Orange rows are decided by fallback, because the rule is silent. Blue rows are decided by stage 2, because the rule fired. Fallback's probability on the blue rows is below 0.5 and is ignored.

The orange rows are training processes whose cross-node all-reduce is missing. Fallback sees the startup allocation and calls them training. That is the feature doing what it was built for. The blue rows are inference with balanced traffic on the wire. Stage 2 sees memory moving, the way it saw training, and calls them training. On every one of the 1,032 windows in this score where the rule fired, stage 2's probability was at least 0.62. It never vetoed.

Encoder utilization, decoder utilization, and the variation of the memory clock are never split on in either forest. Those columns are constant in the training corpus.

Adversarial workloads read off these trees

These are adversarial jobs designed from the interpretation above, not a fresh sample of ordinary workloads and not a rerun of Table 3. The job itself is a training step or an inference server. A spare allocation with no step does not count. The rows we want are the orange ones: real training, placed on the not-training side of a cut the saved trees use, so the detector calls the job not training. That call is this detector's inference call. Blue rows are the other way, real inference placed so the detector calls it training, and they are not the target. None of them has been captured. They are not a reason to move the 0.5 cutoff or to refit either forest. Do not add them to a training set and then score them: a tree that has seen the cell can memorize it. Run each orange row once, against this freeze, to see whether the cut it was aimed at is the cut that decides.

The orange rows are built from two fallback facts in this note. The startup feature reads the first 30 samples of the capture, about 30 seconds, and every later window reuses that same prefix. A jump after sample 30 never enters the feature. Sleeping 40 seconds and then allocating does hide the jump. It also sets time-to-plateau near 40 samples. Fallback cuts that column at 16 samples, and the slow side is training (7 of 200 trees ask it first; the training share above the cut is 0.38 higher than below it). That is a weaker question than the jump, and it points the same way. A sampler that starts only after memory is already flat hides both: there is no jump, and time-to-plateau is 0 because the memory is already at its plateau. Those are different cells.

Two later fallback questions are what the large and small orange rows are set against. Memory utilization minus GPU utilization is cut at about -23 points. The high side is not training. A step that pins the processors and barely touches the memory bus sits on the training side. A model that fills the GPU does not: memory utilization is high too, so the difference is the other side of -23. The 68 GB footprint cut is the same shape. Above it, fallback votes not-training. A small model is on the training side of that cut. Crossing it has to be the training step itself, model plus optimizer state plus activations, sitting between 68 and 78 GB. A buffer allocated beside the step is not a training workload, and it is not one of these rows. The large warm-attach row is that real step, placed on the not-training side of both questions, with the startup jump already hidden.

The flat-counter row is the orange row that still looks like training on the wire. The step is real, the all-reduce is real, and the GPU counters are held on the not-training side of every stage 2 cut in Table 1, so that forest has to veto or the call stays training. The blue rows are real inference, not the target. A serving job loads weights in the first 30 seconds, which is fallback's startup jump, and the wire stays quiet. One of them stays under 68 GB. The other is a real server whose weights and cache sit between 68 and 78 GB, so the jump and the footprint cut point opposite ways. No row is an allocation with no training step and no server.

Cell	Asked	Do this	Called training	Called not training
Warm attach, large	fallback	InfiniBand off. Start the sampler only after memory has been flat for a minute. The training step itself uses 68 to 78 GB. If it does not, drop the row.	The jump was not what caught the orange rows. Name the column that still votes training.	Once the jump is gone, the full-GPU cut is enough. This is the sharpest miss the cuts allow.
Warm attach, small	fallback	Same late sampler, same quiet wire, same kind of step. Plateau under 10 GB. Compare only to the large row.	A small footprint is still enough for fallback, or the processors-versus-memory question is.	Hiding the jump is enough even under 10 GB. The 68 GB cut was not required.
Sleep, then allocate	fallback	Sampler already running. Sleep 40 seconds, then allocate the same small model. InfiniBand off.	The slow plateau replaced the jump. This row is not a clean removal of the startup tell.	Hiding the jump is enough even when time-to-plateau is on the training side.
Flat counters, real all-reduce	stage 2	A real small step. Send+receive at least 1 MiB/s, symmetry at least 0.8. Memory under 10 GB, utilization range under 4.2 points (about 3.4 GB on an 80 GB GPU), power spread under 9 W, plateau inside 16 samples. Fixed clocks, no checkpoint, no sleep.	Either a Table 1 cut was actually crossed, or some other column is carrying the vote. The trace has to say which.	The first stage 2 veto in this score. The 1,032 rule-on windows never went below 0.62.
Ordinary small step	stage 2	Same all-reduce and the same small model. Ordinary clocks, ordinary allocator. Do not hold the counters flat.	A real step does not live on the not-training side of Table 1. Record which cut it crossed.	Small training already gets called not training, with no costume.

Cell	Asked	Do this	Called training	Called not training
Serving, quiet wire	fallback	A real server. Capture from process start. Load weights in the first 30 seconds, then serve. Stay under 68 GB. No cross-node traffic.	A weight load and the optimizer jump look the same to fallback.	Fallback already separates them. The 93 jobs do not show it: their inference false alarms all have the rule on, and fallback was ignored.
Serving past 68 GB	fallback	The same server. Weights and cache themselves sit between 68 and 78 GB. Nothing is allocated beside the server.	The jump wins over the footprint cut.	The footprint cut wins. The two questions in Table 2 that point opposite ways have been put on one job.

Table 4. Adversarial jobs read off Tables 1 and 2. Every row is a real training step or a real server. Orange rows are training aimed at a not-training call. Blue rows are inference aimed at a training call, and they are not the target. "Asked" is the forest that speaks if the wire does what the row says. The last two columns are how to read the call. They are not predictions.

Asymmetric traffic with the jump left in the first 30 samples is a control, not another evasion. Symmetry under 0.8 keeps the rule silent, and fallback should still call the job training, the way it called iboff_flat. If it does not, the jump was not the reason those orange rows were caught.

qwen_fake_ar, olmoe_ep_mixed, and disagg_echo are already the blue rows of Table 3. qwen_one_spike is already a single burst, and only about a tenth of its windows were called training. iboff_flat, nosync_flat, and lora_1n8_iboff are already a visible jump on a quiet wire. None of the rows above is a rerun of those.